Object-Oriented Design Interview

Dmitry Kira

Software Engineer at Google

Interviewing at Google, Facebook, Uber, Airbnb, Lyft, Twitter, Dropbox, Square, Stripe, Pinterest, Quora, Yelp, Zillow, Redfin, Zoom, Slack, Twitch, Tesla, Rivian, Nuro, Waymo, Cruise, Aurora, Zoox, Apple, Microsoft, Amazon

- Enables product selection while preventing additional money insertion to avoid overpayment.
- Validates if the selected product is available and the inserted amount covers the product's cost.
- Transitions to DispenseState upon successful product selection.

DispenseState:

- Represents the state where the vending machine is prepared to dispense the selected product.
- Displays prompts like *"Dispensing product..."* or *"Please collect your change."*
- Handles product dispensing and resets the machine to the initial state after completion.
- Prevents further actions (e.g., inserting money or selecting another product) until the process is complete.

Why does this work?

The State Pattern explicitly defines the transitions between states, ensuring that actions follow the required order. Here's how it works:

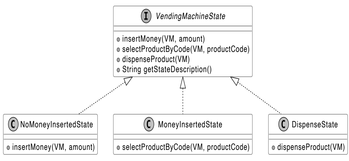
- In NoMoneyInsertedState, users can only insert money. If they try to select a product, the system raises an error.
- In MoneyInsertedState, users must select a product before the system dispenses anything.
- In DispenseState, the vending machine completes the transaction and prevents further actions until it resets.

This approach guarantees the sequence: **Insert Money → Select Product → Dispense Product**.

Each state provides user feedback based on its context:

- NoMoneyInsertedState: "Insert money to proceed."
- MoneyInsertedState: "Select a product."
- DispenseState: "Dispensing product..." or "Please collect your change."

We now define a VendingMachineState interface that serves as a blueprint for the three states (NoMoneyInsertedState, MoneyInsertedState, and DispenseState).



VendingMachineState interface

Code changes

The VendingMachineState interface sets the rules for all states of the vending machine. It includes the behaviors that different states of the vending machine should implement, such as inserting money, selecting products, dispensing products, and describing the current state.

```
public interface VendingMachineState {
    // Handles money insertion in the current state
    void insertMoney(VendingMachine VM, double amount);

    // Handles product selection in the current state
    void selectProductByCode(VendingMachine VM, String productCode)
            throws InvalidStateException;

    // Handles product dispensing in the current state
    void dispenseProduct(VendingMachine VM) throws InvalidStateException;

    // Returns a description of the current state
    String getStateDescription();
}
```

Here is the code for the NoMoneyInsertedState class:

```
public class NoMoneyInsertedState implements VendingMachineState {
    // Adds money to the machine and transitions to MoneyInsertedState
    @Override
    public void insertMoney(VendingMachine VM, double amount) {
        VM.addBalance(amount);
        VM.setState(new MoneyInsertedState());
    }

    // Throws exception as product selection is not allowed without money
    @Override
    public void selectProductByCode(VendingMachine VM, String productCode)
            throws InvalidStateException {
        throw new InvalidStateException("Cannot select a product without inserting money.");
    }

    // Throws exception as product dispensing is not allowed without money
    @Override
    public void dispenseProduct(VendingMachine VM) throws InvalidStateException {
        throw new InvalidStateException("Cannot dispense product without inserting money.");
    }

    // Returns a description of the current state
    @Override
    public String getStateDescription() {
        return "No Money Inserted State - Please insert money to proceed";
    }
}
```

For brevity, the implementation of the MoneyInsertedState and DispenseState classes is omitted, but they follow the same structure.

Wrap Up

In this chapter, we have designed and implemented a Vending Machine system. The most important takeaway from this chapter is how we divided responsibilities across classes, such as Product, Rack, InventoryManager, and PaymentProcessor, while unifying them under a facade for a clear and simple-to-access API. This approach not only simplified the system's external interface but also adhered to the Single Responsibility Principle, ensuring each component focused on a specific responsibility. For instance, the InventoryManager managed stock levels, while the PaymentProcessor handled cash payments and calculated change.

In the deep dive section, we explored state-based control using the State Pattern to enforce a strict sequence of actions and prevent invalid behaviors like dispensing without payment.

In interviews, remember to emphasize validation and error handling after implementing core functionality, especially for systems where improper behavior could cause damage or financial loss.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Further Reading: State Design Pattern

This section gives a quick overview of the design patterns used in this chapter. It's helpful if you're new to these patterns or need a refresher to better understand the design choices.

State design pattern

The State pattern is a behavioral pattern that allows an object to alter its behavior when its internal state changes, making it appear as though the object is behaving like a different class.

In the vending machine design, we use the State pattern to manage states like NoMoneyInsertedState, MoneyInsertedState, and DispenseState, enabling the VendingMachine to switch behaviors dynamically without modifying its core logic.

To illustrate the State pattern in another domain, the following example uses the traffic light system.

Problem

Imagine we have a TrafficLight class. The traffic light can be in one of three states: Red, Yellow, or Green. The behavior of the traffic light changes depending on its current state:

- In the Red state, the light stays red for a set duration.
- In the Yellow state, the light blinks yellow, signaling cars to slow down and prepare to stop.
- In the Green state, the light stays green to allow traffic to pass.

If we were to implement this logic using conditionals, we would need to check the current state of the traffic light every time an action occurs.

While the solution works initially, several issues arise as the system becomes more complex:

Scalability: As the number of states increases, the conditionals grow larger. For example, adding a new state (like a flashing state for emergency vehicles) would require adding more checks to the existing logic, making the code increasingly hard to manage and prone to errors.

Maintainability: The duplication of code and the need to update the same conditional logic in multiple places make the system difficult to maintain over time. This is a crucial problem because it impacts long-term code quality and increases the chance of introducing bugs when modifying the logic.

Solution

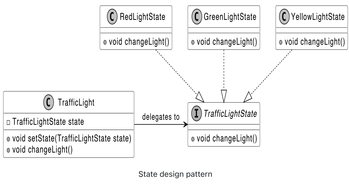
Instead of relying on conditionals to manage state transitions, we can use the State pattern, which encapsulates the behavior associated with each state into separate classes.

Rather than handling all behaviors on its own, the original object, known as the context, holds a reference to one of the state objects that represents its current state, delegating the state-related tasks to that object.

For example, a TrafficLight context can delegate its behavior to a state object, like RedLightState, GreenLightState, or YellowLightState. Each of these states knows how to handle the actions specific to that state, such as changing the light or transitioning to the next state.

To transition to a new state, the context simply replaces the current state object with another one that represents the new state. For instance, when the light is Green, the system transitions to Yellow, and then to Red, without needing complex conditionals.

Here is the representation of the state pattern.



State design pattern

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.